

LOOP — AI Customer-Feedback Intelligence Platform

Close the loop on customer feedback: ingest it from multiple channels, let AI classify and cluster it, see what's trending, and ask plain-English questions grounded in the real data.

Built for the Zidio Internship Web Development track (Project LOOP brief).

Tech stack

Layer	Technology
Framework	Next.js 14 (App Router) + TypeScript
Styling	Tailwind CSS
Database	PostgreSQL
ORM	Prisma
Auth	NextAuth (Credentials provider, JWT sessions)
AI	Anthropic Claude API
Charts	Recharts
Validation	Zod

What's implemented

Core: multi-tenant workspaces, 3 roles (Admin/Analyst/Viewer) enforced server-side, manual + CSV + simulated-channel ingestion, a searchable/filterable paginated inbox with a status workflow, and an analytics dashboard with four charts.

AI (all four, end-to-end on real data):

1. **Auto-classification** — every item gets sentiment, a score, a theme, and a feature area from Claude, stored on ingestion (not recomputed per page load).
2. **Theme clustering & trends** — feedback groups into reused themes; a trends view shows week-over-week spikes.
3. **Ask LOOP** — retrieval-grounded Q&A. The app retrieves the most relevant feedback *before* asking Claude, and the model is instructed to answer only from what it's given.
4. **Voice-of-Customer report** — stats are computed in plain code first; Claude only writes the narrative around numbers that are already correct.

One honest simplification

Real "semantic search" normally means neural embeddings stored in a vector database (pgvector + a hosted embeddings API). To keep this project runnable on any plain Postgres instance with zero extra API keys, `lib/search.ts` builds a term-frequency vector per feedback item and ranks by cosine similarity instead. It's a genuine retrieval step — Ask LOOP really does fetch relevant feedback before answering, and won't fabricate feedback that isn't in the data — it's just not a neural embedding. If you want to extend this, swap `toVector()` for a call to a hosted embeddings provider and store the result in `pgvector`; the rest of the retrieval flow doesn't change.

Local setup

Prerequisites

- Node.js 18+
- A free PostgreSQL database — [Neon](#) or [Supabase](#) both work
- An Anthropic API key
- (For collecting your submission) a Vercel account

Steps

```
# 1. Install dependencies
npm install

# 2. Configure environment variables
cp .env.example .env
# then fill in DATABASE_URL, NEXTAUTH_SECRET, ANTHROPIC_API_KEY

# 3. Push the schema and seed demo data
npx prisma migrate dev --name init
npm run seed

# 4. Run locally
npm run dev
# → http://localhost:3000
```

Generate `NEXTAUTH_SECRET` with:

```
openssl rand -base64 32
```

Demo login credentials (after seeding)

Role	Email	Password
Admin	admin@demo.loop	Demo1234!
Analyst	analyst@demo.loop	Demo1234!
Viewer	viewer@demo.loop	Demo1234!

Architecture

```

Browser (React Server/Client Components)
  |
  ▼
API route handlers (src/app/api/**)
  |   - auth guard (requireUser) + role guard on every route
  |   - every DB query filtered by the caller's workspaceId
  |
  ▼
lib/db.ts (Prisma) —> PostgreSQL
lib/ai.ts           —> Anthropic Claude API (server-side only)
lib/search.ts       —> term-vector retrieval for Ask LOOP

```

Non-negotiable rule enforced throughout: every query touching **Feedback**, **Theme**, **Report**, or **User** is scoped by **workspaceId** from the authenticated session — never from a client-supplied value. See **lib/auth.ts** → **requireUser()** and every route handler in **src/app/api**.

Data model

See **prisma/schema.prisma**. Six entities: **Workspace**, **User**, **Feedback**, **Theme**, **FeedbackTheme** (join table with a confidence score), **Embedding**, and **Report** — every tenant-owned table carries a **workspaceId** foreign key.

Repository structure

```

src/
  app/
    login/, signup/           - public auth pages
    (app)/                    - everything behind the auth guard
    dashboard/ inbox/ trends/ ask/ reports/ settings/
    api/
      auth/ feedback/ themes/ insights/ reports/ members/
    components/                - shared UI (Nav, StatCard, Badge, EmptyState)
  lib/
    auth.ts                   - NextAuth config + requireUser() role/tenant guard
    db.ts                     - Prisma client singleton
    ai.ts                     - all Claude API calls (classify, answer, report)
    search.ts                 - retrieval for Ask LOOP
    csv.ts                    - CSV import parsing

```

```
    validations.ts – Zod schemas  
prisma/  
  schema.prisma  
  seed.ts
```